

Clock V2

What you'll learn/use:

- @State

- Stores data that can change during runtime.
- When updated, SwiftUI refreshes the view automatically.

```
1 //04.09.2025 - AP CSP - ClockV2
2 import SwiftUI
3
4
5 struct ContentView: View {
6     @State private var angle: Double = 0
7     @State private var np: Double = 0
8     @State private var tapCount: Int = 0
9     @State var hour: Int = 0
10    @State var am: Int = 0
11    @State var pm: Int = 0
12    @State var minute: Int = 0
13
14
```

```
67    .frame(width: 300, height: 300)
68
69
70
71    ZStack {
72        Rectangle()
73            .fill(Color.indigo)
74            .frame(width: 8, height: 140)
75            .offset(y: -70)
76            .rotationEffect(.degrees(np))
77
78        Rectangle()
79            .fill(Color.black)
80            .frame(width: 8, height: 90)
81            .offset(y: -45)
82            .rotationEffect(.degrees(angle))
83
84    }
85
86
87
88
89    Circle()
90        .fill(Color.white)
91        .frame(width: 12, height: 12)
92
93    HStack {
94        Button("- hour") {
95            hour = hour - 1
96            angle = angle - 15
97        }
98        .offset(y: 200)
99        .foregroundColor(.white)
100        Button("+ Hour") {
101            hour = hour + 1
102            angle = angle + 15
103        }
104    }
105
```

--ZStack/ HStack / VStack

- **Stacks** arrange elements:
 - ZStack = layers (front-to-back).
 - HStack = horizontal row.
 - VStack = vertical column.
 -

- Color / Shape Views

- Color.orange / Color.gray → background colors.
- Circle() / Rectangle() → geometric shapes.
 - .fill(Color) → fills with color.
 - .frame(width:, height:) → sets size.

-Text("...")

- Displays **text** on screen.
- Modifiers:
 - .font(.title) → changes font size/style.
 - .foregroundColor(.white) → sets text color.
 - .bold() → makes text bold.
 - .offset(y:) → moves text up/down (or x for sideways).

```
Text("hour: \$(hour)")
  .font(.title)
  .foregroundColor(.white)
  .offset(y: -280)
  .bold()

Text("min: \$(minute)")
  .font(.title)
  .foregroundColor(.white)
  .offset(y: -340)
  .bold()
```

```
Rectangle()
  .fill(Color.black)
  .frame(width: 8, height: 90)
  .offset(y: -45)
  .rotationEffect(.degrees(angle))
```

- rotationEffect(.degrees())

- Rotates a shape or view by a certain number of degrees.

- Button("Label") { action }

- Creates a clickable button.
- Inside { } → define what happens when clicked.

```
HStack {
  Button("- hour") {
    hour = hour - 1
    angle = angle - 15
  }
}
```

```
}
.onTapGesture(count: 2) {
  angle += 15
  np += 6
  tapCount += 1
  hour += 1
  if hour == 24 { hour = 0 }
  am += 1
  if am == 12 { am = 0 }
  pm += 1
  if pm == 12 { pm = 0 }
  minute += 1
  if minute == 60 { minute = 0 }
}
```

- .onTapGesture(count:)

- Detects taps on a view.
- count : 2 → triggers only on **double taps**.

- .animation(.easeInOut(duration:), value:)

- Adds a **smooth transition** when a value changes.
- .easeInOut → starts slow, speeds up, then slows down again.
- value: → which variable change triggers the animation.

```
.animation(.easeInOut(duration: 0.3), value: angle)
.animation(.easeInOut(duration: 0.3), value: np)
```

//04.09.2025 - AP CSP - ClockV2

```
import SwiftUI
struct ContentView: View {
    @State private var angle: Double = 0
    @State private var np: Double = 0
    @State private var tapCount: Int = 0
    @State var hour: Int = 0
    @State var am: Int = 0
    @State var pm: Int = 0
    @State var minute: Int = 0

    var isNight: Bool {

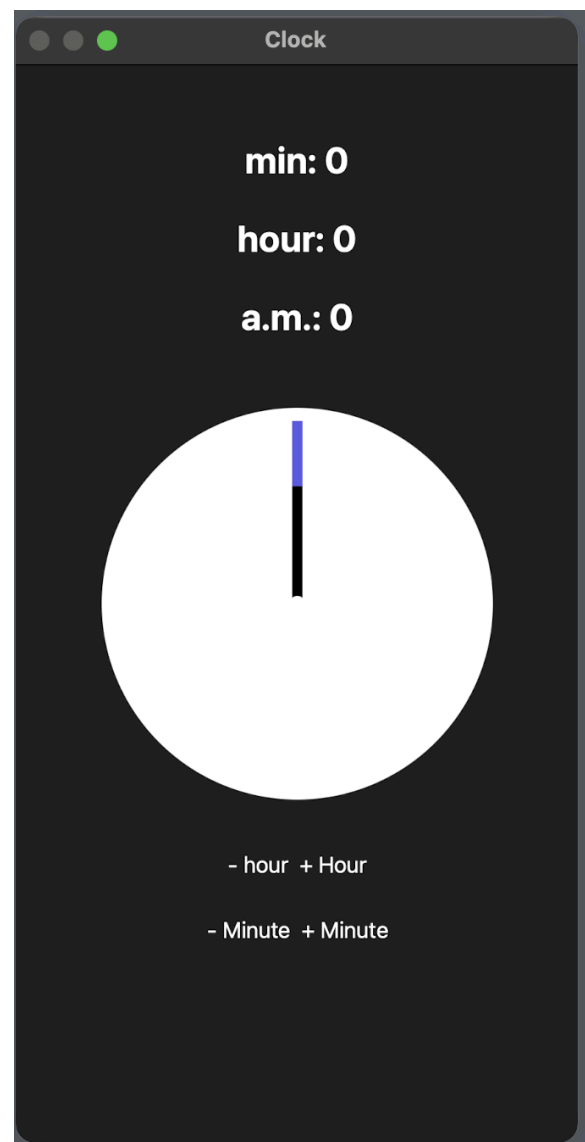
        let mod = tapCount % 24
        return mod < 12
    }

    var body: some View {

        let appledistinguished = isNight ?
        Color.orange : Color.gray
        let educator = isNight ? Text("a.m.") : Text("p.m.")

        ZStack {
            educator
                .ignoresSafeArea()

            appledistinguished
                .offset(y: 280)
                .font(.title)
        }
    }
}
```



```

        .bold()
        .hidden()

Text("hour: \{(hour)")
    .font(.title)
    .foregroundColor(.white)
    .offset(y: -280)
    .bold()

Text("min: \{(minute)")
    .font(.title)
    .foregroundColor(.white)
    .offset(y: -340)
    .bold()

if appledistinguished == Color.orange {
    Text("a.m.: \{(am)")
        .font(.title)
        .foregroundColor(.white)
        .offset(y: -220)
        .bold()
} else {
    Text("p.m.: \{(pm)")
        .font(.title)
        .foregroundColor(.white)
        .offset(y: -220)
        .bold()
}

Circle()
    .fill(Color.white)
    .frame(width: 300, height: 300)

ZStack {
    Rectangle()
        .fill(Color.indigo)
        .frame(width: 8, height: 140)
        .offset(y: -70)
        .rotationEffect(.degrees(np))

    Rectangle()
        .fill(Color.black)
        .frame(width: 8, height: 90)
        .offset(y: -45)
        .rotationEffect(.degrees(angle))
}

```



```

Circle()
  .fill(Color.white)
  .frame(width: 12, height: 12)

HStack {
  Button("- hour") {
    hour = hour - 1
    angle = angle - 15
  }
  .offset(y:200)
  .foregroundColor(.white)
  Button("+ Hour") {
    hour = hour + 1
    angle = angle + 15
  }
  .offset(y:200)
  .foregroundColor(.white)
}
HStack {
  Button("- Minute") {
    minute = minute - 1
    np = np - 6
  }
  .offset(y:250)
  .foregroundColor(.white)
  Button("+ Minute") {
    minute = minute + 1
    np = np + 6
    if minute == 60 { minute = 0 }
  }
  .offset(y:250)
  .foregroundColor(.white)
}
}
.onTapGesture (count: 2) {
  angle += 15
  np += 6
  tapCount += 1
  hour += 1
  if hour == 24 { hour = 0 }
  am += 1
  if am == 12 { am = 0 }
  pm += 1
  if pm == 12 { pm = 0 }
  minute += 1
  if minute == 60 { minute = 0 }
}
.animation(.easeInOut(duration: 0.3), value: angle)
.animation(.easeInOut(duration: 0.3), value: np)
}
}

```

